

Algorithm Design Patterns

Lesson Objectives

By the end of this lesson, students should be able to:

- Understand the major algorithm design strategies: **Divide and Conquer**, **Greedy**, and **Dynamic Programming**.
 - Identify when to apply each approach.
 - Write pseudocode for common examples such as **Coin Change** and **Shortest Path**.
 - Analyze algorithm trade-offs using real-world cases (like game pathfinding).
-

Introduction to Algorithm Design Patterns

Algorithm design patterns are **strategies or techniques** used to solve computational problems efficiently.

Just like engineers reuse design blueprints, programmers reuse **algorithmic strategies** to tackle similar classes of problems.

Common patterns include:

- Divide and Conquer
 - Greedy
 - Dynamic Programming
-

Divide and Conquer

Concept

Break a problem into smaller **subproblems**, solve them **recursively**, and **combine** their results.

Steps:

1. **Divide:** Split the problem into subproblems.
2. **Conquer:** Solve each subproblem recursively.
3. **Combine:** Merge results to get the final answer.

◆ Examples

- Merge Sort
- Quick Sort
- Binary Search

Example: Binary Search (Divide and Conquer)

Goal: Search for a value in a sorted list.

Pseudocode

```
PROCEDURE BinarySearch(A, low, high, key)
  IF low > high THEN
    RETURN -1
  ENDIF
  mid ← (low + high) / 2
  IF A[mid] = key THEN
    RETURN mid
  ELSE IF key < A[mid] THEN
    RETURN BinarySearch(A, low, mid-1, key)
  ELSE
    RETURN BinarySearch(A, mid+1, high, key)
  ENDIF
ENDPROCEDURE
```



Complexity:

$O(\log n)$ — halves the problem each time.

Diagram Idea:

A list split into halves each recursive call:

[1, 3, 5, 7, 9, 11, 13]

→ mid = 7 → search left or right half

3 Greedy Algorithms

💡 Concept

At each step, make the **locally optimal** choice, hoping it leads to a **globally optimal** solution.

✅ Simple

⚠️ Doesn't always guarantee the best overall solution.

♦ Example: Coin Change (Greedy Approach)

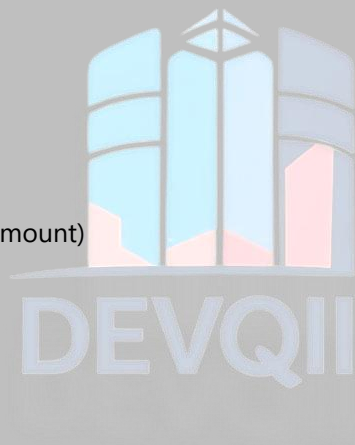
Goal: Make change using the fewest coins.

Available coins: 25, 10, 5, 1

Target: 63

Pseudocode

```
PROCEDURE GreedyCoinChange(amount)
  coins ← [25, 10, 5, 1]
  FOR each c in coins
    WHILE amount ≥ c
      PRINT c
      amount ← amount - c
    ENDWHILE
  ENDFOR
ENDPROCEDURE
```



Output:

25, 25, 10, 1, 1, 1

Works well when coin denominations are canonical (like U.S. currency).

Fails for irregular denominations (e.g., 1, 3, 4).

4 Dynamic Programming (DP)

💡 Concept

Solve complex problems by **breaking them into overlapping subproblems**, solving each **once**, and **storing results** to avoid recomputation.

Used when:

- Subproblems overlap.
- The optimal solution depends on previous results.

◆ Steps

1. Define **subproblems**.
2. Define a **recurrence relation**.
3. Solve using **memoization (top-down)** or **tabulation (bottom-up)**.

◆ Example: Coin Change (DP Approach)

Goal: Find minimum coins to make a target amount.

Pseudocode

```
PROCEDURE MinCoins(amount, coins)
  dp[0] ← 0
  FOR i ← 1 TO amount
    dp[i] ← ∞
    FOR each c in coins
      IF i ≥ c THEN
        dp[i] ← MIN(dp[i], 1 + dp[i - c])
      ENDIF
    ENDFOR
  ENDFOR
  RETURN dp[amount]
ENDPROCEDURE
```

Complexity: $O(n \times m)$
(n = amount, m = number of coin types)

5 Case Study: Pathfinding in Games 🎮

Pathfinding determines the **shortest path** from a character's start point to its destination.

◆ Common Algorithms

Algorithm	Strategy	Description
Dijkstra's	Greedy	Finds shortest path from a start node to all others.
A* (A-star)	Greedy + Heuristic	Uses both path cost and estimated distance to goal.

A Heuristic Formula:*

$$f(n) = g(n) + h(n)$$

- $g(n)$: cost from start to node n
- $h(n)$: estimated cost from n to goal

Pathfinding Diagram Idea

Grid of nodes with arrows showing traversal from **Start** → **Goal**, with the algorithm exploring shortest paths and ignoring obstacles.

Comparing the Three Patterns

Feature	Divide & Conquer	Greedy	Dynamic Programming
Structure	Divide → Solve → Combine	Choose best at each step	Store results of subproblems
Subproblem overlap	None	Possible	Yes
Example	Merge Sort	Coin Change	Fibonacci, Shortest Path
Complexity	$O(n \log n)$	Depends	Often $O(n^2)$ or better
Best for	Independent subproblems	Local optimization	Global optimization

7 Lab Activity

Objective:

Apply each pattern to simple problems.

Task	Design Pattern	Description
Find max element (recursive)	Divide & Conquer	Split array recursively
Make change (coins)	Greedy	Use largest denominations first
Shortest path in a grid	Dynamic Programming	Store minimum path cost

Deliverables:

- Pseudocode for all three problems.
- A short explanation of which pattern fits best and why.

8 Git Task

1. Create a new branch:
2. `git checkout -b design_patterns`
3. Save your pseudocode files:
 - `divide_conquer.txt`
 - `greedy.txt`
 - `dynamic_programming.txt`
4. Add and commit:
5. `git add .`
6. `git commit -m "Added design pattern pseudocode examples"`
7. `git push origin design_patterns`